INVERTIS UNIVERSITY, BAREILLY

B.Tech CSE – Third Year

APPLIED DATA STRUCTURES & ALGORITHMS LAB (BCS553)

LAB 12

Assignment 12: Recursive Factorial, Sum and Power

Assignment aim: Solve numeric problems using recursive methods and base cases.

Experiment No. 12.1 — Recursive factorial

1. Experiment Title: Recursive factorial

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Recursion solves a problem by calling the same method on a smaller input until a base case is reached.

5. Algorithm / Procedure:
Base case: factorial(0) = 1; otherwise return n multiplied by factorial(n - 1); evaluate factorial(5). Then compile and execute the Java program.

6. Program:

```
public class Task_01_Recursive_factorial{
    static int factorial(int n) {
        if (n == 0) return 1;
        return n * factorial(n - 1);
    }
    public static void main(String[] args) {
        System.out.println(factorial(5));
    }
}
```

7. Input: Fixed input values are declared in the program: factorial(5), sum(10), or power(2, 5), as applicable.

8. Output:
120

Time / Space Complexity: O(n) time; O(n) call-stack space.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- What is recursion?
- What is the base case?
- Why is 0! equal to 1?

Experiment No. 12.2 — Recursive sum from 1 to n

1. Experiment Title: Recursive sum from 1 to n

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Recursion solves a problem by calling the same method on a smaller input until a base case is reached.

5. Algorithm / Procedure:
Return zero when n is zero; otherwise add n to sum(n - 1); print sum(10).
Then compile and execute the Java program.

6. Program:

```
public class Task_02_Recursive_sum_from_1_to_n{
    static int sum(int n) {
        if (n == 0) return 0;
        return n + sum(n - 1);
    }
    public static void main(String[] args) {
        System.out.println(sum(10));
    }
}
```

7. Input: Fixed input values are declared in the program: factorial(5), sum(10), or power(2, 5), as applicable.

8. Output:
55

Time / Space Complexity: O(n) time; O(n) call-stack space.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- How does the recursive call get smaller?
- What is sum(0)?
- What is the sum from 1 to 10?

Experiment No. 12.3 — Recursive power calculation

1. Experiment Title: Recursive power calculation

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Recursion solves a problem by calling the same method on a smaller input until a base case is reached.

5. Algorithm / Procedure:
Return 1 when exponent is zero; otherwise multiply base by power(base, exp - 1); print 2 to the power 5.
Then compile and execute the Java program.

6. Program:

```
public class Task_03_Recursive_power_calculation{
    static int power(int base, int exp) {
        if (exp == 0) return 1;
        return base * power(base, exp - 1);
    }
    public static void main(String[] args) {
        System.out.println(power(2, 5));
    }
}
```

7. Input: Fixed input values are declared in the program: factorial(5), sum(10), or power(2, 5), as applicable.

8. Output:
32

Time / Space Complexity: O(exp) time; O(exp) call-stack space.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- What is the base case?
- Why is any nonzero base to exponent 0 equal to 1?
- How many recursive levels are used for exp = 5?